

DOCKER

INTRODUCTION

par

Marek Goldmann

10 Déc 2013



docker

DOCKER ?

Un projet pour créer et gérer des conteneurs.

CONTENEURS ?

Oui, une virtualisation de système d'exploitation légère.

QUELLE COMPARAISON PAR RAPPORT À UNE MACHINE VIRTUELLE?

C'est **complètement** différent.

COMPARAISON

	Machine virtuelle (KVM, VMware)	Conteneur (LXC, OpenVZ)
Matériel	Simulé	L'utilise (presque) en direct
OS supportés	Presque tous	Seulement Linux
Espace	Espace utilisateur	Espace noyau
Séparation	Complète	Control Groups (cgroups)
Temps démarrage	Secondes à minutes	Milisecondes
Passage échelle	Quelques une	Infini (milliers)
Noyau spécifique	Oui	Non
Fonctions avancées (migration live, etc)	Oui	Non
Facilité de création	Moyenne	Facile
Durée requise à création	Longue	Courte
Taille	ENORME	Petite

QUI EST DERRIÈRE DOCKER?

dotCloud* et une **communauté puissante.**

* dotCloud, Inc. a été **renommée en Docker, Inc.** récemment

EN PARLANT DE LA COMMUNAUTÉ...

- **GitHub**
 - 501 watches
 - 6948 étoiles
 - 847 forks
 - 406 tickets ouverts
 - 69 pull requests
- Plus de **200 contributeurs**
 - en moy. 13 commits / jour
 - **92% sont externes**
- De nombreuses rencontres partout dans le monde
 - **15** groupes de **rencontres** actifs

Atteint seulement après **10 mois** depuis le premier commit sur GitHub

GLOSSAIRE

- **Image** : un *package* avec un filesystem et des données
- **Conteneur** : une *image* en fonctionnement
- **Registry** : endroit où les *images* sont stockées/partagées

CONCEPTS PRINCIPAUX

- Une image peut être liée à une image **parente**
- Les images sans parent sont appelées image de **base**
- Un conteneur a un état, alors que les images n'en ont pas
- Vous pouvez lancer de multiples conteneurs à partir d'une image

COMPOSANTS DE DOCKER

MANAGER

Fait fonctionner, démarre, arrête, etc... sur les conteneurs

BUILDER

Crée les images

REGISTRY DES IMAGES

Stockage pour les images

CRÉER DES IMAGES DOCKER

C'est facile ! On utilise des **Dockerfiles**.

DOCKERFILE

Un fichier **texte** avec des instructions qui **automatisent** la fabrication de l'image.

EXEMPLE DE DOCKERFILE

```
# Se base sur une image de Fedora 20
FROM mattdm/fedora

# Met à jour l'image
RUN yum -y update

# Installe java
RUN yum -y install java-1.7.0-openjdk-devel wget

# Récupère WildFly
RUN wget http://download.jboss.org/wildfly/8.0.0.Beta1/wildfly-8.0.0.Beta1.tar.gz -O /
RUN tar -xf /opt/wildfly.tar.gz -C /opt

# Lance WildFly au démarrage du conteneur
ENTRYPOINT /opt/wildfly-8.0.0.Beta1/bin/standalone.sh -c standalone-ha.xml -b 0.0.0.0
```

Ceci devrait être sauve dans le fichier appelé `Dockerfile`

INSTRUCTIONS DE BASE

- **FROM** : l'image sur laquelle on se base / qu'on étend
- **RUN** : La commande à exécuter à la fabrication
- **ENTRYPOINT** : La commande à exécuter après le démarrage du conteneur

Mais il y en a **beaucoup d'autres** !

FABRICATION DE L'IMAGE

`docker build .`

FABRICATION DE L'IMAGE

```
$ docker build .  
Uploading context 10240 bytes  
Step 1 : FROM mattdm/fedora  
  
[SNIP]  
  
Step 4 : RUN wget -q http://download.jboss.org/wildfly/8.0.0.Beta1/wildfly-8.0.0.Beta1  
----> Running in 41467b97b423  
----> 112766c23202  
Step 5 : RUN tar -xf /opt/wildfly.tar.gz -C /opt  
----> Running in d07d0c5c3dab  
----> c53f9f4ceb1e  
Step 6 : ENTRYPOINT /opt/wildfly-8.0.0.Beta1/bin/standalone.sh -c standalone-ha.xml -b  
----> Running in a9b2cecf4db5  
----> 5894688e4f2e  
Successfully built 5894688e4f2e
```

Nous avons fabriqué l'image 5894688e4f2e

MODIFICATION DE L'IMAGE

1. Changer le Dockerfile
2. Exécuter la commande `build` à nouveau

PARLONS DE RAPIDITÉ

```
$ time docker build .  
Uploading context 10240 bytes  
Step 1 : FROM mattdm/fedora  
----> 0f3e92b4e94d  
  
[SNIP]  
  
Step 4 : RUN wget -q http://download.jboss.org/wildfly/8.0.0.Beta1/wildfly-8.0.0.Beta1  
----> Using cache  
----> 2ad6655478dd  
Step 5 : RUN tar -xf /opt/wildfly.tar.gz -C /opt  
----> Using cache  
----> c0f63f06e9ec  
Step 6 : RUN touch /etc/something  
----> Running in 8426dd5570af  
----> 7ef016a00cde  
Step 7 : ENTRYPOINT /opt/wildfly-8.0.0.Beta1/bin/standalone.sh -c standalone-ha.xml -b
```

Nous avons construit **une nouvelle** image e757e0ea05d7

Oui, **300ms!**

COMMENT ÇA MARCHE

1. Préparation de l'exécution d'une instruction
 - Est-ce que l'instruction a déjà été exécutée auparavant ?
 - **oui** - utiliser le **résultat précédent**, aller au #3
 - **non** - exécuter l'instruction, aller au #2
2. Est-ce que l'instruction s'est exécutée sans erreur ?
 - **oui** - **commit**, aller au #3
 - **non** - interrompre le build
3. Est-ce que c'est la fin des instructions ?
 - **oui** - **commit**, fini !
 - **non** - aller au #1

PARLONS DE TAILLE

```
$ docker images
```

REPOSITORY	TAG	ID	CREATED	SIZE
<none>	<none>	e757e0ea05d7	37 minutes ago	12.29
<none>	<none>	5894688e4f2e	46 minutes ago	12.29

Oui, **12.29kB!**

RÉFÉRENTIELS D'IMAGES

1. **Privé** : par exemple hébergé à l'intérieur d'une entreprise
2. **Public** : index.docker.io

COMMANDES DOCKER CLI

DOCKER PULL

Récupérer une image ou un repository dans le registry

DOCKER PUSH

Transmettre une image depuis le registry vers un repository

DOCKER RUN

Lance une commande dans un **nouveau conteneur** basé sur l'image spécifiée

DOCKER STOP

Arrête un conteneur

DOCKER START

Démarre un conteneur arrêté

DOCKER RM

Supprime un conteneur arrêté

DOCKER RMI

Supprime une image du registry

OK, SUPER, **MAIS...**

CAS D'UTILISATION ?

- Voulez-vous votre propre **PaaS**?
- Tests automatisés et **CI**
- Deployer des applis **qui passent à l'échelle**
- **plateforme de déploiement** d'applications

PAAS

Il est (relativement) facile de créer une PaaS avec Docker
Il y a même un **tutoriel** pour en créer une en 5 minutes...

OPENSIFT

<https://github.com/openshift/openshift-pep/blob/master/openshift-pep-010-docker-cartridges.md>

INTÉGRATION CONTINUE (CI)

1. Vous committez
2. la CI construit l'appli
3. la CI crée une image avec votre nouvelle appli
4. la CI démarre un conteneur à partir de cette nouvelle image
5. les tests d'intégration sont exécutés sur une application **réelle**
6. **Bénéfice !**

APPLIS PASSANT À L'ÉCHELLE

- **Prérequis** : préparer son appli pour qu'elle passe facilement à l'échelle
- Démarrer un conteneur est rapide et pas cher
- Étendre un cluster (par exemple WildFly) devient facile
- **Bénéfice !**

PLATEFORME DE DÉPLOIEMENT

Fabriquer une fois - exécuter n'importe où !

- Télécharger facilement et rapidement
- Mettre à jour les images est également facile

BÉNÉFICE !

PAR OÙ COMMENCER ?

```
yum install docker-io
```

[**http://docs.docker.io/en/latest/use/basics/**](http://docs.docker.io/en/latest/use/basics/)

MERCI!



Remerciements :

<https://www.docker.io/>

